UNIVERSITY OF LEEDS

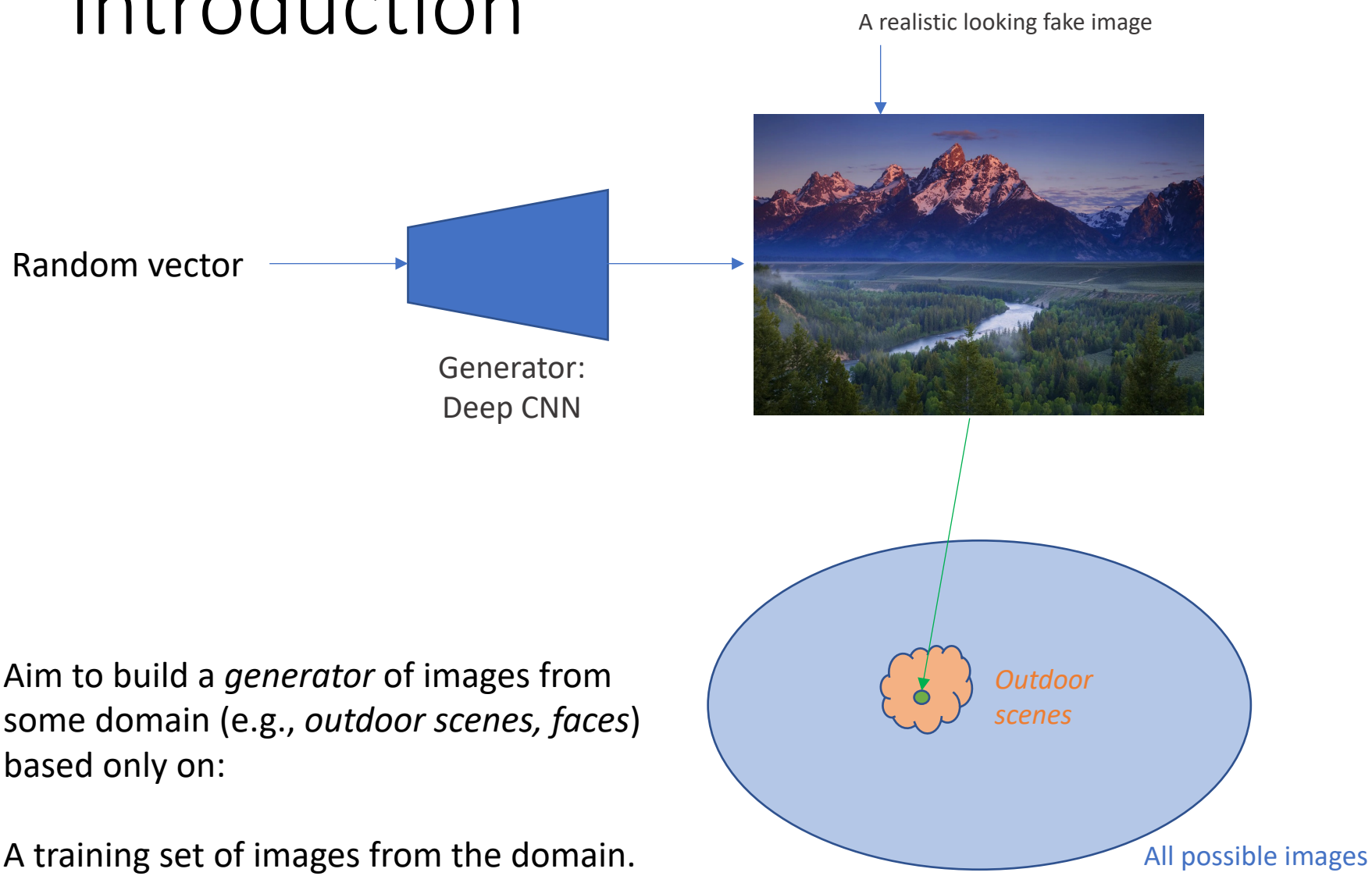
Generative Models

Deep Learning - COMP5203M

2022-23

© The University of Leeds 2021

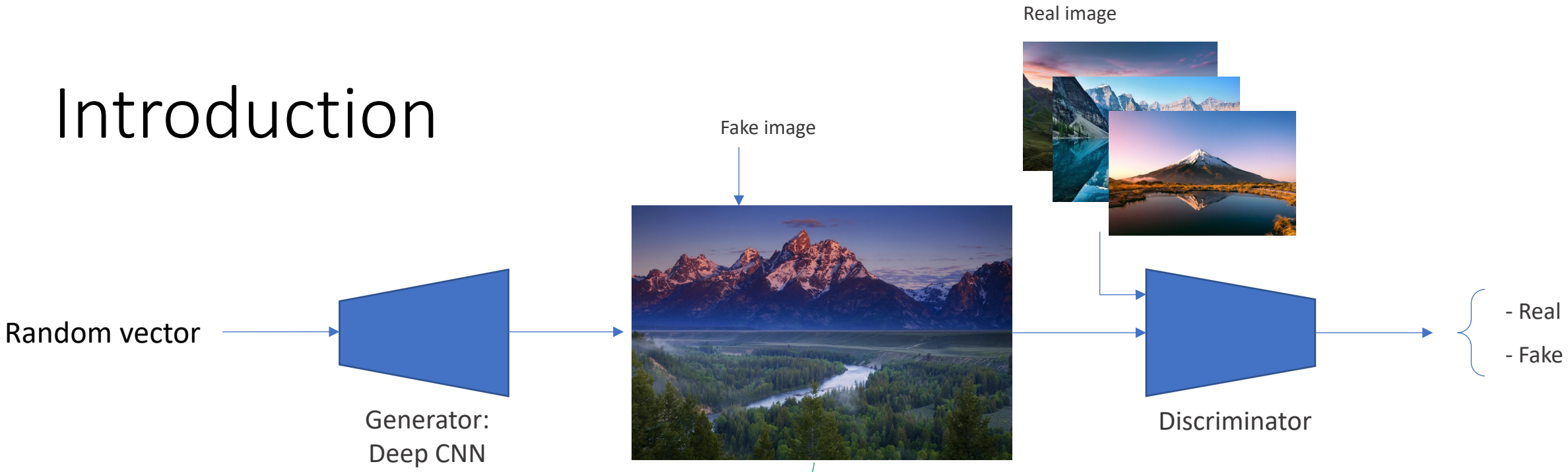
Introduction

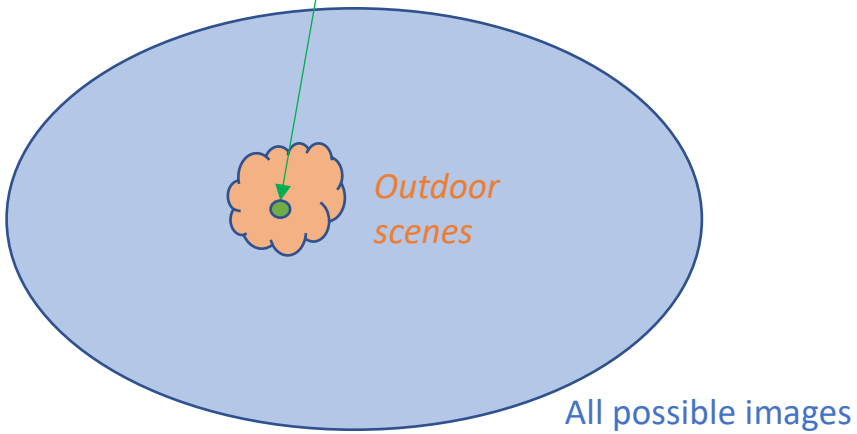


Aim to build a *generator* of images from some domain (e.g., *outdoor scenes, faces*) based only on:

A training set of images from the domain.

Introduction





Aim to build a *generator* of images from some domain (e.g., *outdoor scenes, faces*) based only on:

A training set of images from the domain.

Input vectors

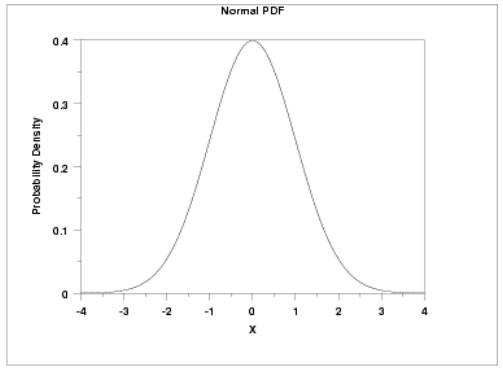
Input vectors are samples drawn from some distribution over $\mathbb{R}^n$, where n is typically between 100 and 512.

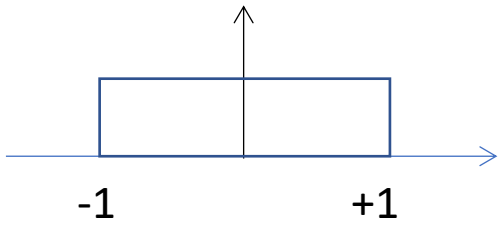
The vectors are not observed (i.e., they are not specified in our dataset) and are therefore referred to as *latent*.

In recent work on image generation, the distribution decomposes so that the n components of each sample are drawn independently from the same 1-D probability distribution. Two common choices for this distribution:

$z \sim \mathcal{N}(0,1)$ normal (Gaussian) distribution with mean 0 and standard deviation 1

$z \sim U(-1,1)$ uniform distribution between -1 and +1

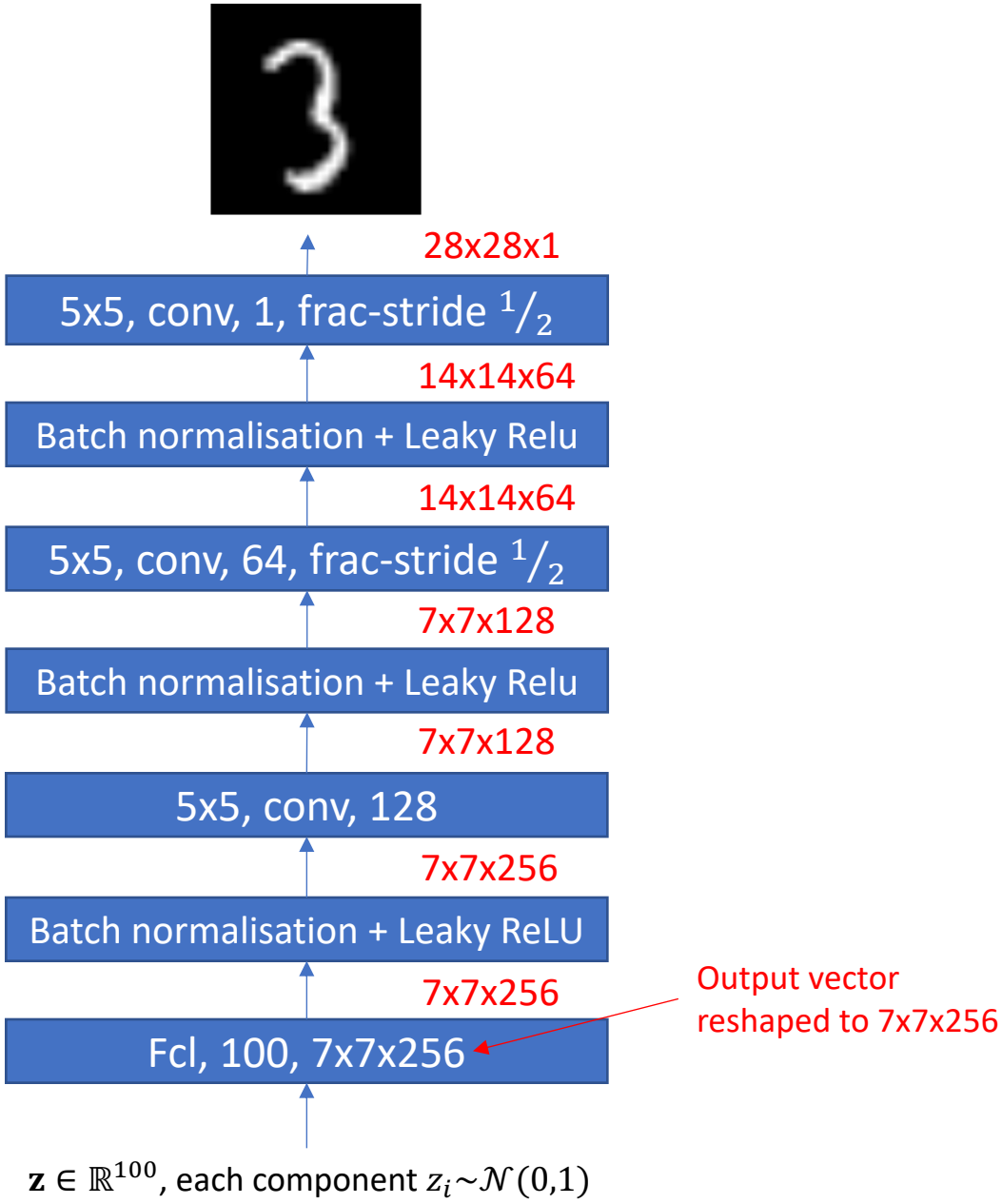




Example - DCGAN

In experiments, the architecture of DCGAN varies according to the target size determined by the source dataset.

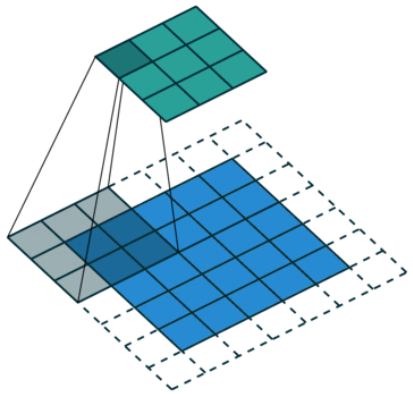
Here is the architecture of DCGAN used for training on the MNIST dataset, where the target size is 28x28x1.



Radford et al., Unsupervised representation learning with deep convolutional generative adversarial networks. ICLR, 2016

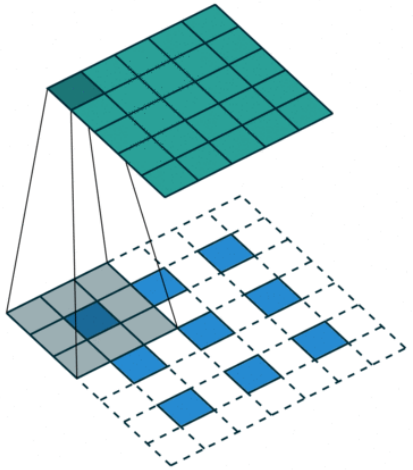
Fractionally-strided convolution

Convolution (padded), with a stride of n, makes output n times smaller than input.



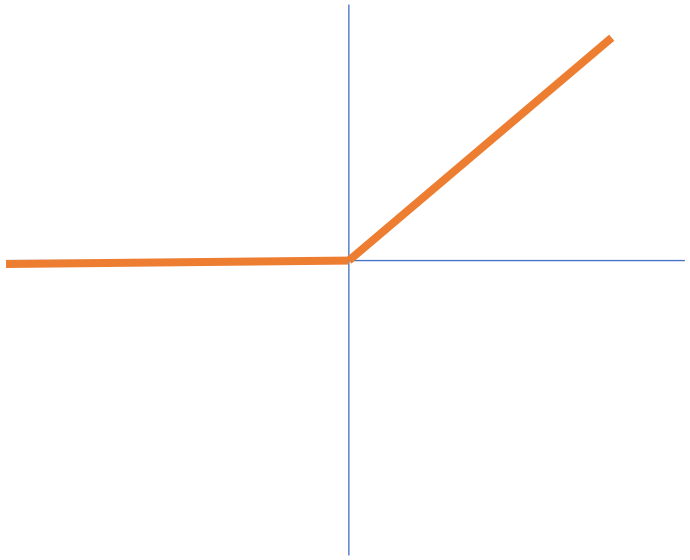
3x3 convolution with stride of 2, zero padding

Convolution with a *fractional-stride* of $\frac{1}{n}$, makes the output n times larger by simply inserting (n-1) rows and columns of zeros between the rows and columns of the input.

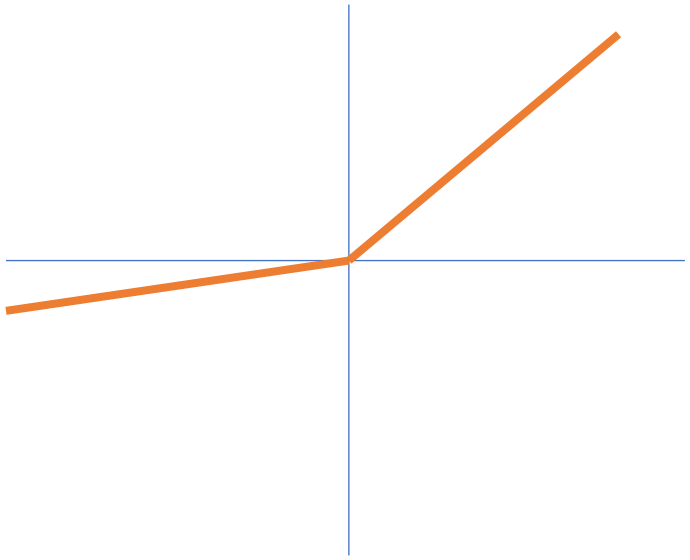


3x3 convolution with fractional stride of $\frac{1}{2}$

Leaky ReLU



ReLU



Leaky ReLU

Close to ReLU but avoids backpropagation of zero gradients

Constructing a loss function for the generator

There is no paired data of the form$\{(\boldsymbol{z}_i, \boldsymbol{x}_i)\}$, allowing supervised learning.

All we have is a collection of images from some domain $\{\boldsymbol{x}_i\}$. How to define a loss?

Solution: use a CNN classifier (*discriminator)* to produce a loss for training the generator.

The discriminator outputs a probability $d(\boldsymbol{x})$ that a given image x is real.

Thus, $1 - d(\boldsymbol{x})$ is the probability that $\boldsymbol{x}$ is fake.

Then use cross-entropy with target 'real' for all fake images.

Updating the discriminator

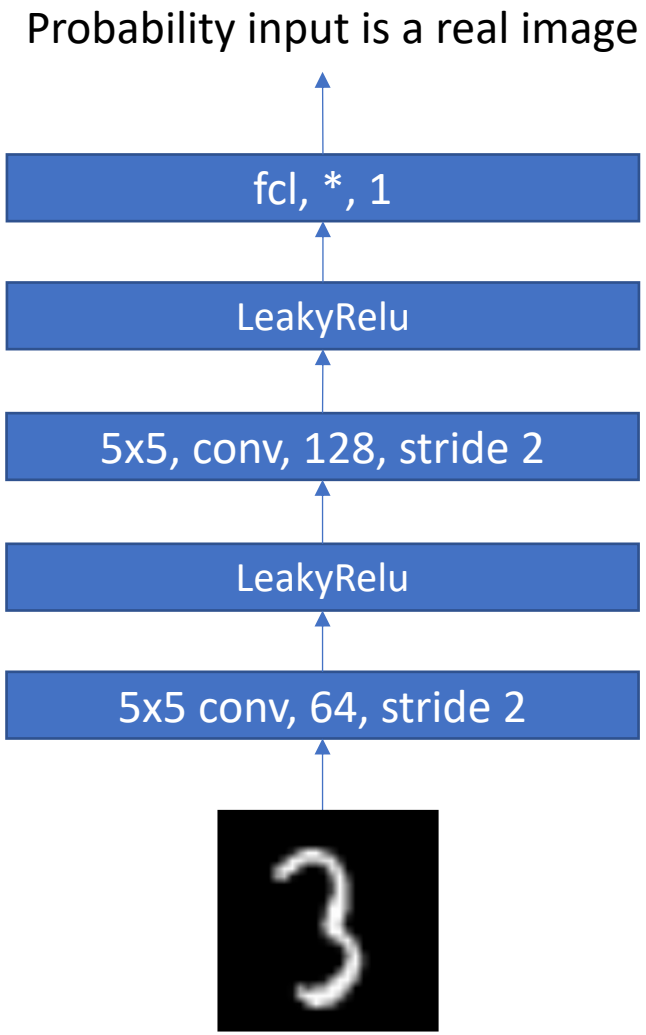
Update the discriminator parameters θ_d from a minibatch of:

- m real images from the chosen domain $\{\boldsymbol{x}^{(1)}, \cdots, \boldsymbol{x}^{(m)}\}$
- m fake images from the generator $\{g(\boldsymbol{z}^{(1)}), \cdots, g\left(\boldsymbol{z}^{(m)}\right)\}$ obtained from random latent vectors $\{\boldsymbol{z}^{(1)}, \cdots, \boldsymbol{z}^{(m)}\}$.

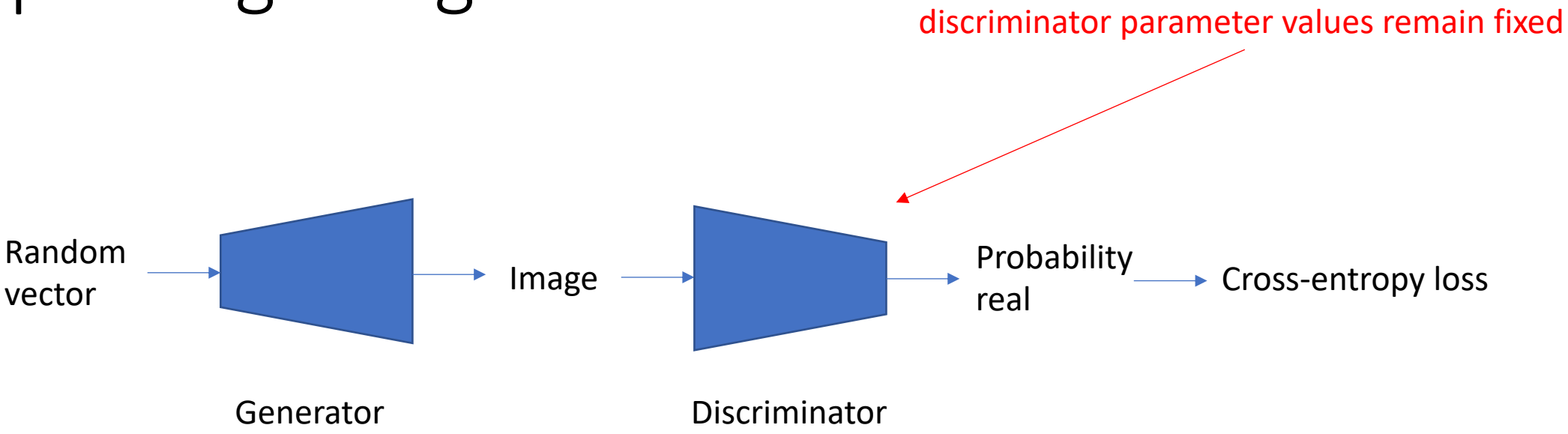
Log likelihood to be maximised: $$\frac{1}{m}\sum_{i=1}^{m} \log(d\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - d\left(g\left(\boldsymbol{z}^{(i)}\right)\right)\right)$$

Standard supervised setting with cross-entropy loss on two-class (real/fake) output distribution.

DCGAN discriminator



Updating the generator



Log likelihood to be maximised: $$\frac{1}{m}\sum_{1}^{m}\log\Big(\underbrace{d\left(g\left(\boldsymbol{z}^{(i)}\right)\right)}_{\text{probability a fake image is real}}\Big)$$

Linked training

The key insight is to train the generator and discriminator iteratively.
Thus, the generator and discriminator improve together, minibatch by minibatch.

Initialise the generator and discriminator networks randomly from a normal distribution
Repeat num_epoch times
Repeat on each batch of real images

Sample from latent distribution and use generator to produce a batch of fake images

Update ***discriminator*** to increase the log likelihood on the output from real and fake images (cross-entropy loss)

Pass fake images through the ***current discriminator*** to generate output probabilities

Backpropagate gradient through discriminator and generator, and update ***generator*** to increase the log likelihood of 'real' for the fake images.

Adversarial setting

Generator and Discriminator can be viewed as adversaries in a zero-sum game. Hence *Generative Adversarial Network.*

Payoff to the discriminator given by:

estimated as average of $\log(d(\boldsymbol{x})$ for a batch of real images

$$v(\boldsymbol{\theta}_g, \boldsymbol{\theta}_d) = \mathbb{E}_{x\sim p_{data}} \log\big(d(\boldsymbol{x})\big) + \mathbb{E}_{x\sim p_{model}} \log(1 - d(\boldsymbol{x}))$$

estimated as average of $\log(1 - d(\boldsymbol{x}))$ for a batch of fake images

Where $\boldsymbol{\theta}_g$ and $\boldsymbol{\theta}_d$ are the parameter values of generator and discriminator.

Payoff to the generator is $1 - v(\boldsymbol{\theta}_g, \boldsymbol{\theta}_d)$.

Making alternate updates to parameter values, discriminator and generator seek to maximise their payoff.

The game continues until we get an optimal generator g^* that has the smallest of the maximum payoffs achieved by the discriminator:

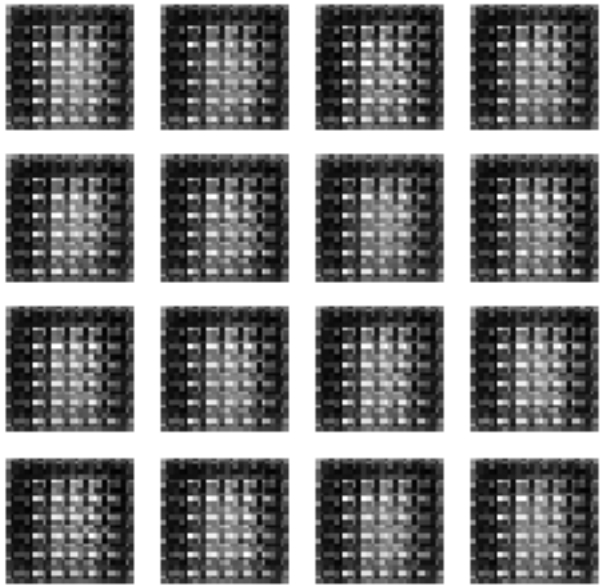
$$g^* = \arg min_g \; max_d \, v(g, d)$$

In practice, updating the generator to increase $1 - v(\boldsymbol{\theta}_g, \boldsymbol{\theta}_d)$ doesn't work as well as seeking to increase:

$$\mathbb{E}_{x \sim p_{model}} \log\big(d(\boldsymbol{x})\big)$$

Experiments with DCGAN

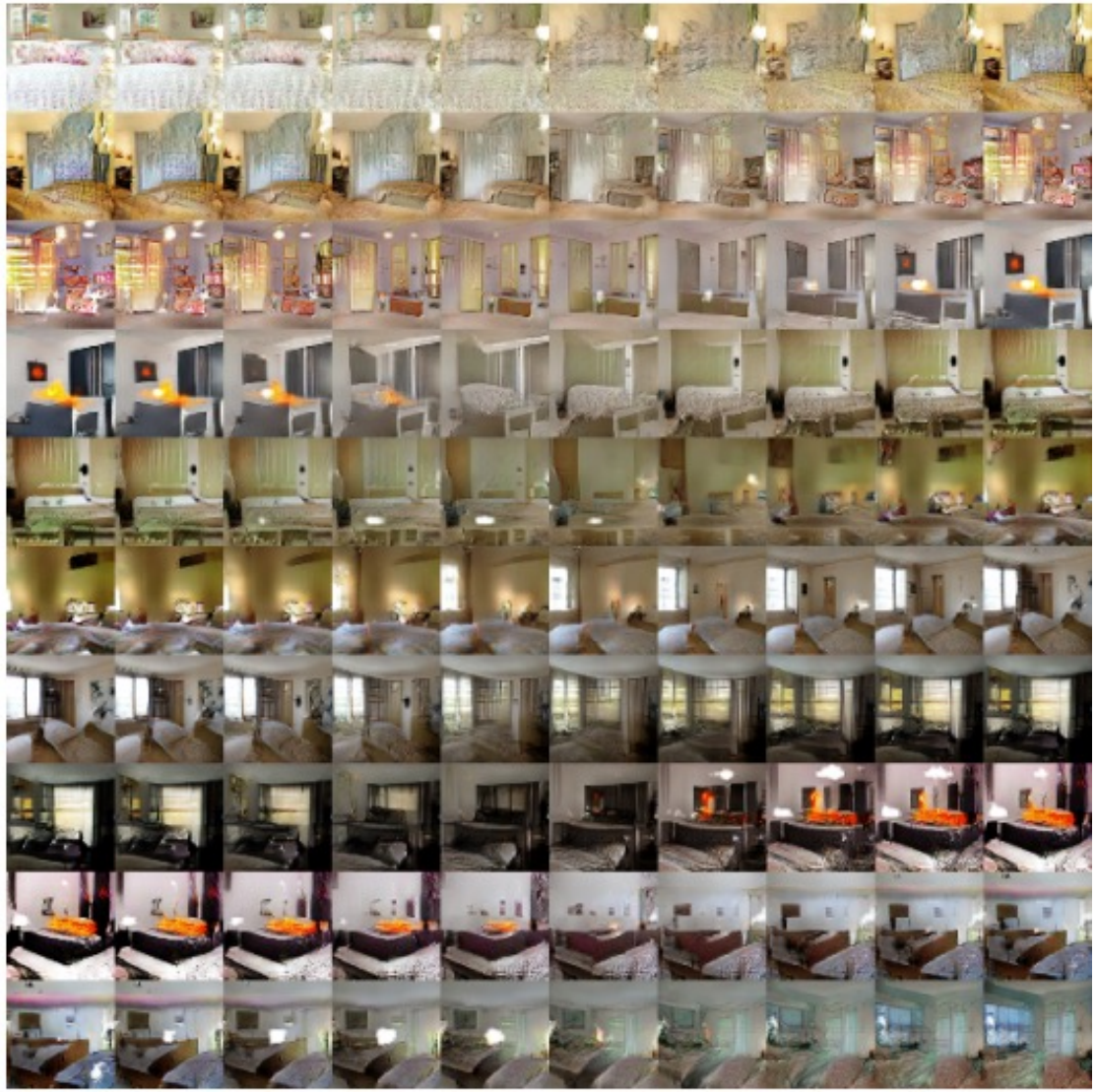
Training on MNIST over 50 epochs



From https://www.tensorflow.org/tutorials/generative/dcgan

Fake images from a generator trained on 3 million images of bedrooms

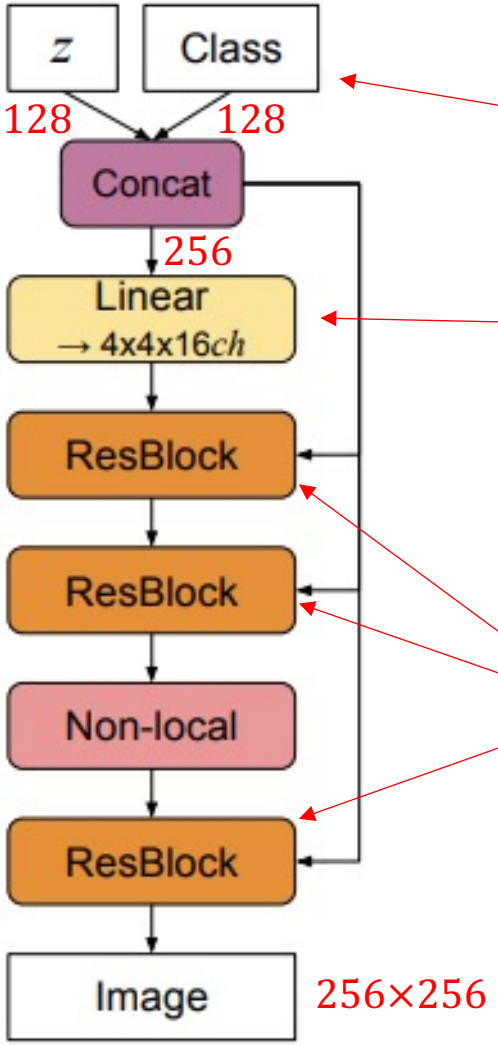
Tours of the output space



Sample nine random input vectors, move between these, generating fake images at equal steps.

BigGAN-deep

Generative model conditioned on a class label



$\mathbf{z} \in \mathbb{R}^{128}$ each component sampled from normal (Gaussian) distribution

Embedding of class label (size 128)

Linear mapping, reshaped into 4x4x16

Residual blocks with multiple convolutional layers, upsampling and batch normalisation.

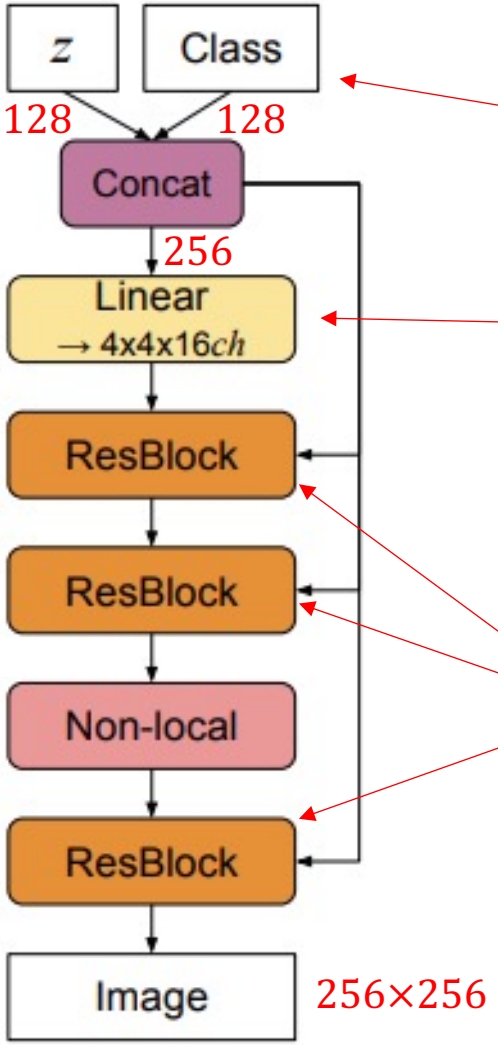
Goes beyond regular fully-connected layer

Brock et al., Large Scale GAN Training for High Fidelity Natural Image Synthesis, ICLR 2019

BigGAN-deep

Generative model conditioned on a class label

$\mathbf{z} \in \mathbb{R}^{128}$ each component sampled from normal (Gaussian) distribution

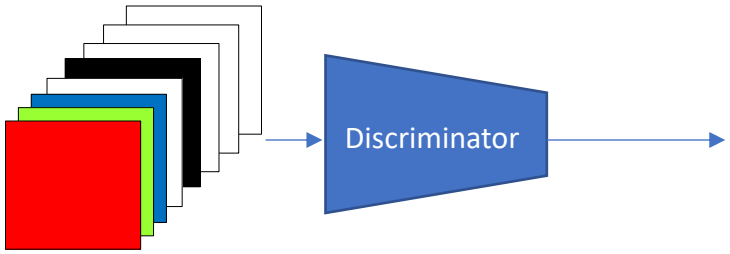


Embedding of class label (size 128)

Linear mapping, reshaped into 4x4x16

Residual blocks with multiple convolutional layers, upsampling and batch normalisation.

Goes beyond regular fully-connected layer



Brock et al., Large Scale GAN Training for High Fidelity Natural Image Synthesis, ICLR 2019

Some generated images

Training on ImageNet
256x256 resolution generated images

UNIVERSITY OF LEEDS

Image-to-image translation

Deep Learning - COMP5203M

2022-23

© The University of Leeds 2021

Zhu et al. Unpaired image-to-image translation using cycle-consistent adversarial networks, ICCV 2017.

Focus on *style transfer* from a photo to a stylised painting (e.g., in the style of Monet)

Two image domains: photos X and stylised paintings Y represented by:

A set of photos $\{\boldsymbol{x}\}_{i=1}^{N} \subset X$

A set of paintings $\{\boldsymbol{y}\}_{i=1}^{M} \subset Y$

Domains X and Y can be seen as probability distributions $p_{data}(x)$ and $p_{data}(y)$

CycleGAN uses two convolutional neural networks $G: X \to Y$ and $F: Y \to X$

If we had the desired paintings from a set of photos, we could use a supervised approach to train G.

Similarly for F.

But we don't have this.

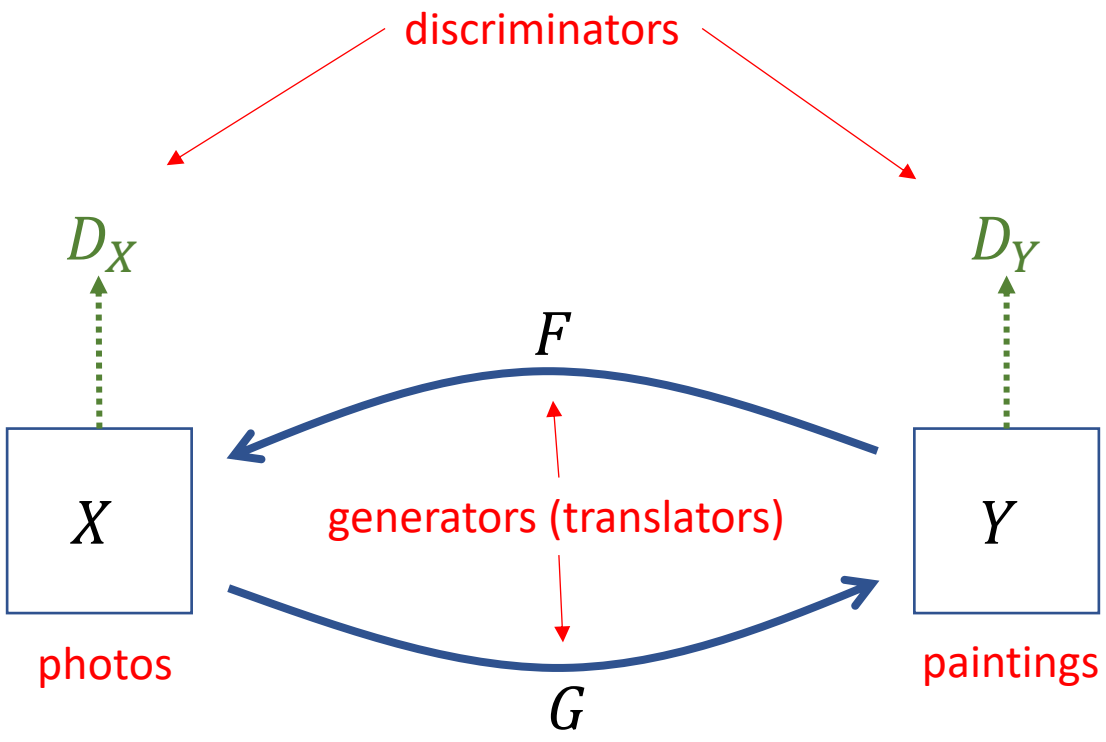
So, use an adversarial approach.

Use two discriminators:

D_Y to measure the realism of paintings generated by G

D_X to measure the realism of photos generated by F

CycleGAN



F, G are convolutional neural networks

Repeat on each batch of real photos and paintings

Use G and F to generate the corresponding fake paintings and photos

Update D_Y to increase the log likelihood of correct targets for G, estimating:

$$\mathcal{L}_{GAN}(G, D_Y, X, Y) = \mathbb{E}_{\boldsymbol{y}\sim p_{data}(\boldsymbol{y})} \log\big(D_Y(\boldsymbol{y})\big) + \mathbb{E}_{\boldsymbol{x}\sim p_{data}(\boldsymbol{x})} \log\Big(1 - D_Y\big(G(\boldsymbol{x})\big)\Big)$$

Update $D_X$to increase the log likelihood of correct targets for F, estimating:

$$\mathcal{L}_{GAN}(F, D_X, X, Y) = \mathbb{E}_{\boldsymbol{x}\sim p_{data}(\boldsymbol{x})} \log\big(D_X(\boldsymbol{x})\big) + \mathbb{E}_{\boldsymbol{y}\sim p_{data}(\boldsymbol{y})} \log\Big(1 - D_X\big(F(\boldsymbol{y})\big)\Big)$$

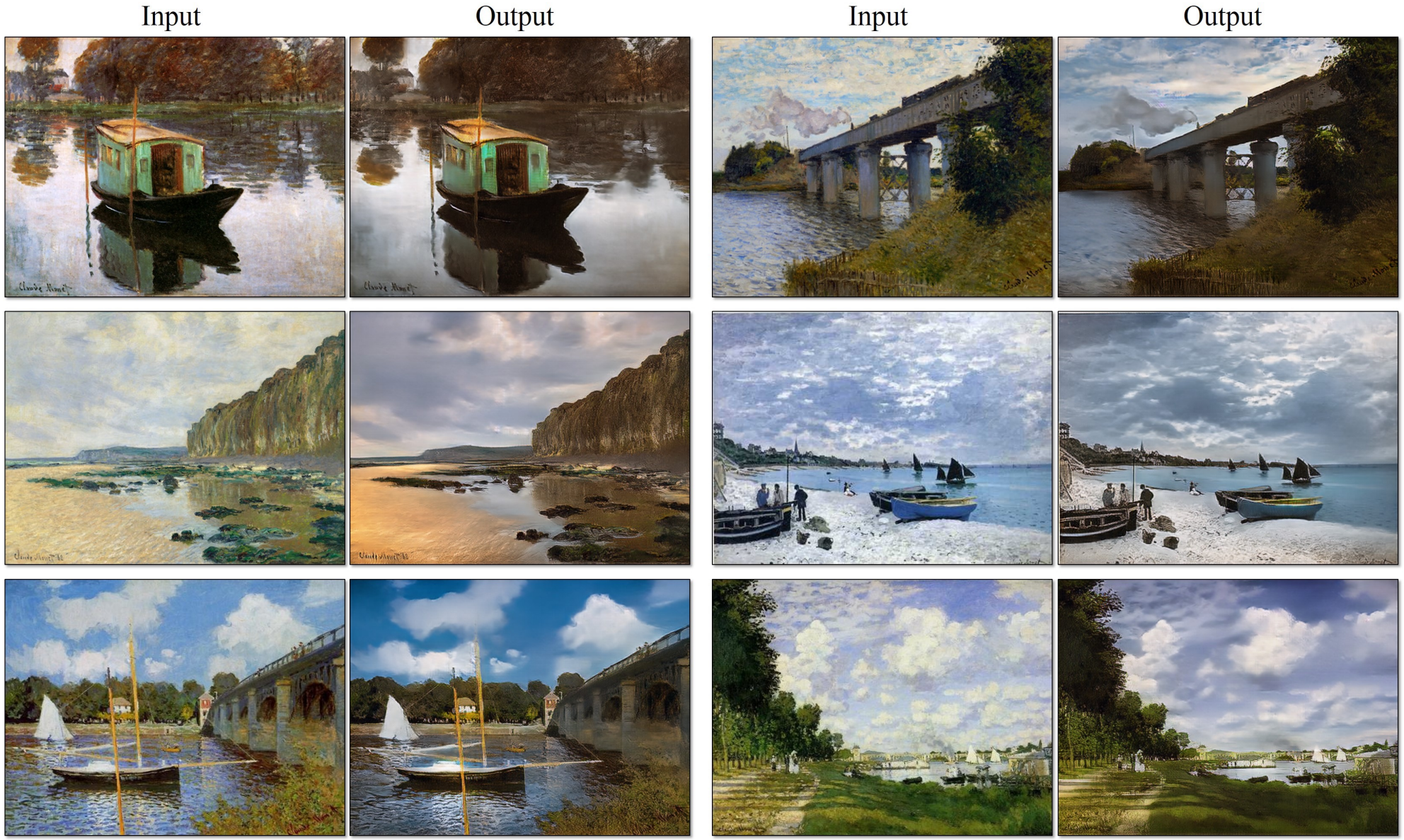
Update G and F together to increase the sum of:

log probability of fake paintings according to DY	$\mathbb{E}_{\boldsymbol{x}\sim p_{data}(\boldsymbol{x})} \log\Big(D_Y\big(G(\boldsymbol{x})\big)\Big) +$
log probability of fake photos according to DX	$\mathbb{E}_{\boldsymbol{y}\sim p_{data}(\boldsymbol{y})} \log\Big(D_X\big(F(\boldsymbol{y})\big)\Big) -$
similarity between roundtrip paintings	$(\mathbb{E}_{\boldsymbol{y}\sim p_{data}(\boldsymbol{y})} \big\|G\big(F(\boldsymbol{y})\big) - \boldsymbol{y}\big\|_1 +$
similarity between roundtrip photos	$\mathbb{E}_{\boldsymbol{x}\sim p_{data}(\boldsymbol{x})} \big\|F\big(G(\boldsymbol{x})\big) - \boldsymbol{x}\big\|_1)$

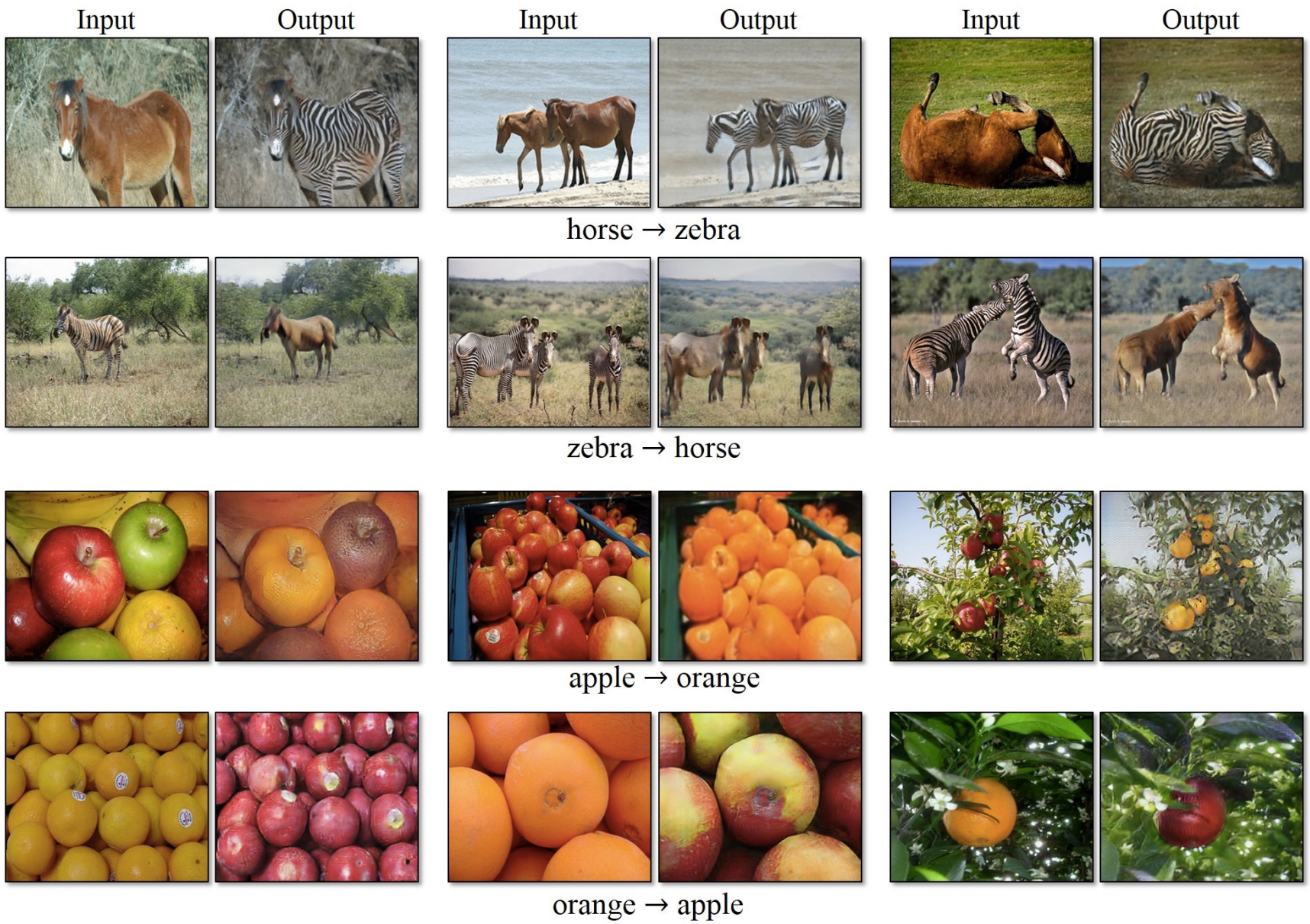
Instead of maximising log likelihood, minimise squared deviation from 1 or 0.

Thus, for example:

$$\mathbb{E}_{\boldsymbol{x}\sim p_{data}(\boldsymbol{x})} \log\Big(D_Y\big(G(\boldsymbol{x})\big)\Big) \quad \text{becomes} \quad \mathbb{E}_{\boldsymbol{x}\sim p_{data}(\boldsymbol{x})}\big(D_Y\big(G(\boldsymbol{x})\big) - 1\big)^2$$



Zhu, J.-Y. et al., ICCV 2017



Zhu, J.-Y. et al., ICCV 2017